SPANS-02: Querying Spans with DQL

> **Series:** SPANS — Distributed Tracing and Spans | **Notebook:** 2 of 8 | **Created:** December 2025 | **Last Updated:** 10/12/2026

Mastering Span Queries in Dynatrace

This notebook covers essential techniques for querying and filtering span data to find exactly what you need. You'll learn to filter by service, operation, and attributes to quickly locate relevant traces.

Table of Contents

1. DQL is NOT SQL!
2. Filtering by Service
3. Filtering by Span Kind
4. Filtering by Operation Name
5. String Matching Functions
6. Finding Specific Traces
7. HTTP Span Queries
    - RPC and gRPC Span Queries
8. Database Span Queries
9. Working with NULL Values
10. Combining Multiple Filters
11. Request Attributes on Spans

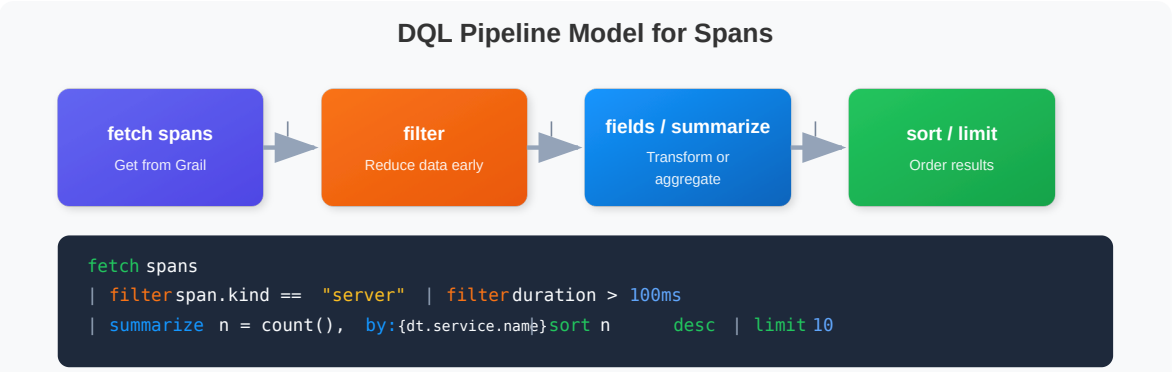
Prerequisites

Before starting this notebook, ensure you have:

- ✅ Completed **SPANS-01: Fundamentals**
- ✅ Access to a Dynatrace environment with span data
- ✅ Permission to read spans (`storage:spans:read`)

1. DQL is NOT SQL!

⚠️ **CRITICAL:** DQL has different syntax from SQL. Memorize these differences:



Key Differences

Concept	SQL	DQL
Arrays	`('a', 'b')` parentheses	`{"a", "b"}` curly braces
Comparison	`=` single equals	`==` double equals
String quotes	`'single quotes'`	`"double quotes"`
NULL checks	`IS NULL` / `IS NOT NULL`	`isNull()` / `isNotNull()`
Membership	`IN (...)`	`in(field, {...})`
Grouping	`GROUP BY`	`by: {...}` in summarize

2. Filtering by Service

Filter spans to focus on specific services. Use `dt.service.name` for the service name and `dt.smartscape.service` for the service ID.

> 💡 **Tip:** On the validation tenant both were set on every span in an hour (322,799 of 322,799), while `service.name` — the OpenTelemetry attribute — was set on 6.7%. `dt.entity.service` still carries the same ID but is `deprecated` in the semantic dictionary.
>
> **Dictionary:** `dt.service.name` (`stable`; the dictionary describes it as equal to the Smartscape service node name); `dt.smartscape.service` (`stable`); `service.name` (`stable`); `dt.entity.service` (`deprecated`), read 10/02/2026.

```
// Filter spans for a specific service using exact match
fetch spans, from:-1h
| filter dt.service.name == "checkout"
| fields start_time, span.name, span.kind, duration
| sort start_time desc
| limit 50
```

```
// Use in() with curly braces {} for multiple values (NOT parentheses!)
fetch spans, from:-1h
| filter in(dt.service.name, {"checkout", "payment", "cart"})
| fields start_time, dt.service.name, span.name, span.kind, duration
| sort start_time desc
| limit 100
```

```
// Count spans by service ID
fetch spans, from:-1h
| filter isNotNull(dt.smartscape.service)
| summarize {span_count = count()}, by: {dt.smartscape.service}
| sort span_count desc
| limit 20
```

3. Filtering by Span Kind

Filter spans based on their role in the distributed transaction.

⚠️ **IMPORTANT:** `span.kind` values are **lowercase**!

Kind	Description	Use Case
`"server"`	Handles incoming request	Find inbound API calls
`"client"`	Makes outgoing request	Find calls to dependencies
`"internal"`	Internal processing	Find business logic
`"producer"`	Sends async message	Find message publishers
`"consumer"`	Receives async message	Find message consumers

```
// Find all SERVER spans (inbound requests)
// Note: "server" is lowercase, not "SERVER"
fetch spans, from:-1h
| filter span.kind == "server"
| fields start_time, dt.service.name, span.name, duration
| sort duration desc
| limit 50
```

```
// Find CLIENT spans (outbound calls to dependencies)
fetch spans, from:-1h
| filter span.kind == "client"
| fields start_time, dt.service.name, span.name, duration
| sort duration desc
| limit 50
```

```
// Count spans by kind to understand your traffic patterns
fetch spans, from:-1h
| summarize {span_count = count()}, by: {span.kind}
| sort span_count desc
```

4. Filtering by Operation Name

Find spans for specific operations or endpoints using the `span.name` attribute:

```
// Find spans for a specific operation/endpoint
fetch spans, from:-1h
| filter contains(span.name, "checkout")
| fields start_time, dt.service.name, trace.id, span.name, duration,
span.status_code
| sort start_time desc
| limit 50
```

```
// Find all POST operations (write operations)
fetch spans, from:-1h
| filter startsWith(span.name, "POST")
| fields start_time, dt.service.name, span.name, duration, span.status_code
| sort start_time desc
| limit 50
```

4.1. Endpoint Names (Enhanced Endpoints)

`span.name` is whatever the instrumentation set — a templated route for some frameworks, a literal path for others — so it is a poor key for "which endpoint was slow". **Endpoints** are Dynatrace's answer: the semantic dictionary describes `endpoint.name` as derived from endpoint detection rules and detected exclusively on request root spans. **Enhanced endpoints for SDv1** brings that endpoint model, and per-endpoint `dt.service.request.*` metrics, to services detected with Service Detection v1.

Environment created in	Enhanced endpoints available in	Default	Configurable?
Dynatrace 1.329 and earlier	1.333+	Off	Yes — *Settings → Process and contextualize → Services → Service detection v1*
Dynatrace 1.330 – 1.332	1.330+	On	Yes
Dynatrace 1.333+	1.330+	On	No — *"always on"*

Practical impact on DQL queries:

Without Enhanced Endpoints	With Enhanced Endpoints
Unmarked requests aggregated into one `NON_KEY_REQUEST` bucket	Each detected endpoint gets its own `dt.service.request.*` metrics
No endpoint field to group by on SDv1 services	`endpoint.name` on request root spans, named from `http.route` where the framework provides it

Filtering rule of thumb:
- Group and filter by `endpoint.name` on request root spans (`isNotNull(endpoint.name)`) when you mean "endpoint". Use `span.name` for the raw operation name.
- On `span.name`, prefer `contains()` / `startsWith()` on the path stem over equality with a literal URL — span names differ by framework and instrumentation.
- For services without the `http.route` span attribute (often Nginx, Apache, IIS in front of a backend), endpoints may collapse to `GET /*`. Use request-naming rules to recover named endpoints — see Enhanced endpoints for SDv1 (DT docs).

> **Changed (SaaS 1.346 — staged rollout from 08/25/2026): fewer `/*` endpoints on SDv2.** Verbatim: *"SDv2 now derives a `http.route` in cases where OneAgent sent the fallback value of `/*`."* On services detected by **SDv2**, endpoints that previously collapsed to `GET /*` may now carry a real derived route — which is the outcome you want, and also a change in the data your saved queries match against. Two things to check once it reaches your tenant: queries or dashboards that **filter for `/*`** as a proxy for "unnamed endpoint" will match fewer rows (that population is shrinking by design, not disappearing), and any **request-naming rule you added purely to work around `/*`** may now be redundant or, worse, competing with a derived route. The `isNull(http.route)` diagnostic in §10 remains the right way to find genuinely missing routes. SDv1 services are unaffected.
>
> **Following on (SaaS 1.347 — staged rollout from 09/16/2026): those derived routes now participate in matching.** Calculated `http.route` values are included in URL path pattern-matching conditions, so a request-naming or service-detection rule written against a URL path can now match on a route that OneAgent never sent. Combined with the 1.346 derivation above, a rule that matched nothing while endpoints collapsed to `/*` can start matching without anyone editing it — review URL path pattern conditions for intended behaviour rather than assuming they are inert.

Services not affected: external services, background activity, queue listeners, key-value stores — Enhanced Endpoints does not create endpoints for these.

> **Dictionary:** `endpoint.name` (`stable`), read 10/02/2026. **Sources:** Enhanced endpoints for SDv1 (DT docs), What's new in Dynatrace SaaS 1.346 (DT docs) — the derived `http.route` change quoted above.

---

5. String Matching Functions

DQL provides several string matching functions:

Function	Description	Example
`contains(field, "text")`	Substring match	`contains(span.name, "user")`
`startsWith(field, "text")`	Prefix match	`startsWith(span.name, "GET")`
`endsWith(field, "text")`	Suffix match	`endsWith(url.path, ".json")`
`matchesPhrase(field, "pattern")`	Wildcard pattern	`matchesPhrase(span.name, "GET /api/*")`
`in(field, {"a", "b"})`	Multiple values	`in(span.kind, {"server", "client"})`

```
// Contains - partial match anywhere in the string
fetch spans, from:-1h
| filter contains(span.name, "Get")
| fields span.name, dt.service.name
| dedup span.name
| limit 20
```

```
// startsWith and endsWith - prefix/suffix matching
fetch spans, from:-1h
| filter startsWith(span.name, "GET") or endsWith(span.name, "query")
| fields span.name
| dedup span.name
| limit 20
```

```
// matchesPhrase - wildcard pattern matching with *
fetch spans, from:-1h
| filter matchesPhrase(span.name, "GET /api/*")
| fields span.name
| dedup span.name
| limit 20
```

```
// Use dedup to see unique span names per service
fetch spans, from:-1h
| filter span.kind == "server"
| fields dt.service.name, span.name
| dedup dt.service.name, span.name
| sort dt.service.name asc
| limit 50
```

---

6. Finding Specific Traces

Locate all spans belonging to a specific trace:

```
// First, find some trace IDs to work with
fetch spans, from:-1h
| filter span.kind == "server"
| fields start_time, trace.id, span.name, dt.service.name
| sort start_time desc
| limit 10
```

```
// Find all spans for a specific trace ID
// Replace YOUR_TRACE_ID with an actual trace.id from above
fetch spans, from:-1h
// trace.id is a uid: keep the toUid() wrapper — a plain string never matches.
// | filter trace.id == toUid("YOUR_TRACE_ID")
| fields start_time, span.id, span.parent_id, span.name, dt.service.name,
duration
| sort start_time asc
| limit 100
```

```
// Find root spans (entry points) - spans without a parent
fetch spans, from:-1h
| filter isNull(span.parent_id)
| fields start_time, trace.id, span.name, dt.service.name, duration,
span.status_code
| sort start_time desc
| limit 50
```

---

7. HTTP Span Queries

Query HTTP-specific span attributes for API troubleshooting:

Attribute	Description
`http.request.method`	HTTP method (GET, POST, etc.)
`http.response.status_code`	HTTP status code (200, 404, 500)
`http.route`	URL route pattern (use this, not url.path for aggregation)
`url.path`	Full URL path (may contain PII)

```
// Query HTTP spans with response status codes
fetch spans, from:-1h
| filter isNotNull(http.response.status_code)
| fields start_time,
         dt.service.name,
         http.request.method,
         http.route,
         http.response.status_code,
         duration
| sort start_time desc
| limit 100
```

```
// Find HTTP 5xx errors (server errors)
fetch spans, from:-1h
| filter http.response.status_code &gt;= 500
      and http.response.status_code &lt; 600
| fields start_time,
         dt.service.name,
         http.request.method,
         http.route,
         http.response.status_code,
         span.status_message,
         duration
| sort start_time desc
| limit 50
```

```
// Summarize HTTP status codes by route
fetch spans, from:-1h
| filter isNotNull(http.response.status_code)
| summarize {status_count = count()}, by: {http.response.status_code}
| sort http.response.status_code asc
```

7a. RPC and gRPC Span Queries

Not every service-to-service call is an HTTP request. RPC frameworks — gRPC above all, plus Java RMI and Apache Dubbo — carry their own span attributes, modelled in the semantic dictionary as the `rpc` model on the `spans` data object.

Core `rpc.*` Attributes

Attribute	Description
`rpc.system`	RPC framework. **Values are not a fixed enum** — see the warning below
`rpc.service`	Fully-qualified service name (e.g. `checkout.v1.CheckoutService`)
`rpc.method`	Method invoked on that service (e.g. `PlaceOrder`)
`rpc.namespace`	Namespace the service is registered under

Attribute	Description
`rpc.grpc.status_code`	gRPC-specific numeric status code (`0` = `OK`)
`server.address` / `server.port`	Callee address, same as for HTTP spans

`span.kind` still tells you which side of the call you are looking at — `server` for the callee, `client` for the caller — so pair it with `rpc.service` / `rpc.method` the same way you pair it with `http.route`.

> **Read `rpc.service` as the interface, not the Dynatrace service.** `rpc.service` is the gRPC service definition from the `.proto` file. It is not `service.name`, and one Dynatrace service commonly exposes several `rpc.service` values.
>
> ⚠️ **`rpc.system` is instrumentation-supplied, so never filter it with `==`.** The value is whatever the instrumenting library wrote, not a value the platform normalizes. A single tenant surveyed on 07/30/2026 carried **`grpc` and `gRPC` simultaneously** — 110,097 and 8,391 spans in the same hour — alongside `adk`, `apache_axis`, `jersey`, and even bare `1` and `2`. Filtering `rpc.system == "grpc"` there returns a confidently wrong number, quietly missing 7% of gRPC traffic.
>
> Use **`matchesValue(rpc.system, "grpc")`** instead: it is an exact match but **case-insensitive**, so it catches every capitalization without also matching unrelated frameworks the way `contains()` could. Before trusting any `rpc.system` filter in your own tenant, look at what is actually there:
>
> ```
> > fetch spans, from:-1h > | filter isNotNull(rpc.system) > | summarize spans = count(), by:{rpc.system} > | sort spans desc >
> ```

```
// Summarize gRPC calls by service and method.
//
// Use matchesValue() rather than == on rpc.system. matchesValue() is an exact
but
// CASE-INSENSITIVE match, and real tenants carry more than one spelling: on
the
// validation tenant both `grpc` (110,097 spans/hr) and `gRPC` (8,391
spans/hr) are
// present, because the value is whatever the instrumentation set. So
// `rpc.system == "grpc"` silently drops ~7% of gRPC traffic — a wrong number,
not an
// error. Live-verified 07/30/2026: matchesValue returns 118,393 spans across
both.
fetch spans, from:-1h
| filter matchesValue(rpc.system, "grpc")
| summarize {
    call_count = count(),
    error_count = countIf(span.status_code == "error"),
    avg_duration_ms = avg(duration) / 1ms
  }, by: {span.kind, rpc.service, rpc.method}
| fieldsAdd error_rate_pct = (error_count * 100.0) / call_count
| sort call_count desc
| limit 50
```

gRPC status codes and the OneAgent version

OneAgent 1.343+. OneAgent 1.343 (released 07/28/2026) adds **gRPC status-code support for .NET**. Rollout is **per fleet, not per tenant** — the tenant version is not the agent version, and agent fleets routinely lag a sprint or more. **Confirm the OneAgent version on the hosts concerned** before you build alerting on `rpc.grpc.status_code` for .NET services.

Until 1.343 reaches those hosts, .NET gRPC spans are **still captured** — you filter on `span.status_code` and the `rpc.*` attributes above, exactly as this section shows. What is missing is only the gRPC-specific status code. So on a mixed fleet, treat a null `rpc.grpc.status_code` as "not yet instrumented on this host", not as a healthy call: `0` means `OK`, and null means you do not know.

The query below separates those two populations so you can see which is which.

```
// gRPC status-code coverage: populated vs. not-yet-instrumented.
// A null rpc.grpc.status_code is NOT a healthy call — it means the gRPC-specific code
// was not populated. Check the OneAgent version on the reporting host.
//
// Case-insensitive rpc.system match for the reason given in the previous cell.
// Live-verified 07/30/2026: of 118,380 gRPC spans in one hour, only 11 carried
// rpc.grpc.status_code — so on a real fleet the "not yet instrumented" population is
// the overwhelming majority, and reading null as OK would be badly wrong.
fetch spans, from:-1h
| filter matchesValue(rpc.system, "grpc")
| summarize calls = count(), by: {rpc.grpc.status_code, span.status_code}
| sort calls desc
| limit 50
```

Feature-flag context on spans

OneAgent 1.343+. OneAgent 1.343 also adds **OpenFeature SDK support for Java**, enriching traces with feature-flag evaluation data. The value is attribution rather than novelty: a latency or error difference can be tied to the flag *variant* that produced it, instead of inferred from deploy timing and a change log. As with gRPC status codes, this ships per agent fleet — **verify the OneAgent version on the Java hosts concerned.**

The pre-1.343 working path is unchanged and stays valid: set the flag variant yourself as a custom span attribute at instrumentation time (see **OTEL-04 § 5 — Span Attributes and Events**) and query it like any other business attribute. That approach is also the portable one — it works on any runtime and any agent version, and it is what you fall back to for languages the platform-native enrichment does not cover.

Either way, keep the variant a **flat, low-cardinality scalar** (`checkout-v2` , `control`) so it works as a `by:{...}` dimension.

---

8. Database Span Queries

Analyze database operations captured as spans:

Attribute	Description
`db.system`	Database type (mysql, postgresql, redis)
`db.namespace`	Database name (was `db.name`)
`db.operation.name`	Operation type (SELECT, INSERT, UPDATE) (was `db.operation`)
`db.query.text`	The database query — may contain sensitive data (was `db.statement`)
`db.collection.name`	Table or collection (was `db.mongodb.collection` and similar)

The older names in brackets are the pre-1.0 OpenTelemetry database conventions. They are null on Grail spans, so a filter on them matches nothing without raising an error.

```
// Field names corrected 08/12/2026 — pre-1.0 OpenTelemetry database semconv
names had been
// used throughout, and every one of them is null on Grail spans. They fail
SILENTLY: a filter on a
// non-existent field matches nothing and a summarize groups everything under
null, so these cells
// returned empty or single-null-group results without ever erroring.
//   db.operation         -&gt; db.operation.name    (stable; set on 50,379 of
57,295 db spans)
//   db.statement         -&gt; db.query.text        (stable; 33,463)
//   db.mongodb.collection-&gt; db.collection.name   (stable; 6,912)
//   db.name              -&gt; db.namespace         (stable; 57,281)
// Confirm the catalog for your tenant with:
//   fetch dt.semantic_dictionary.fields | filter startsWith(name, "db.") |
fields name, stability
// Find all database spans
fetch spans, from:-1h
| filter isNotNull(db.system)
| fields start_time,
         dt.service.name,
         db.system,
         db.namespace,
         db.operation.name,
         duration
| sort duration desc
| limit 50
```

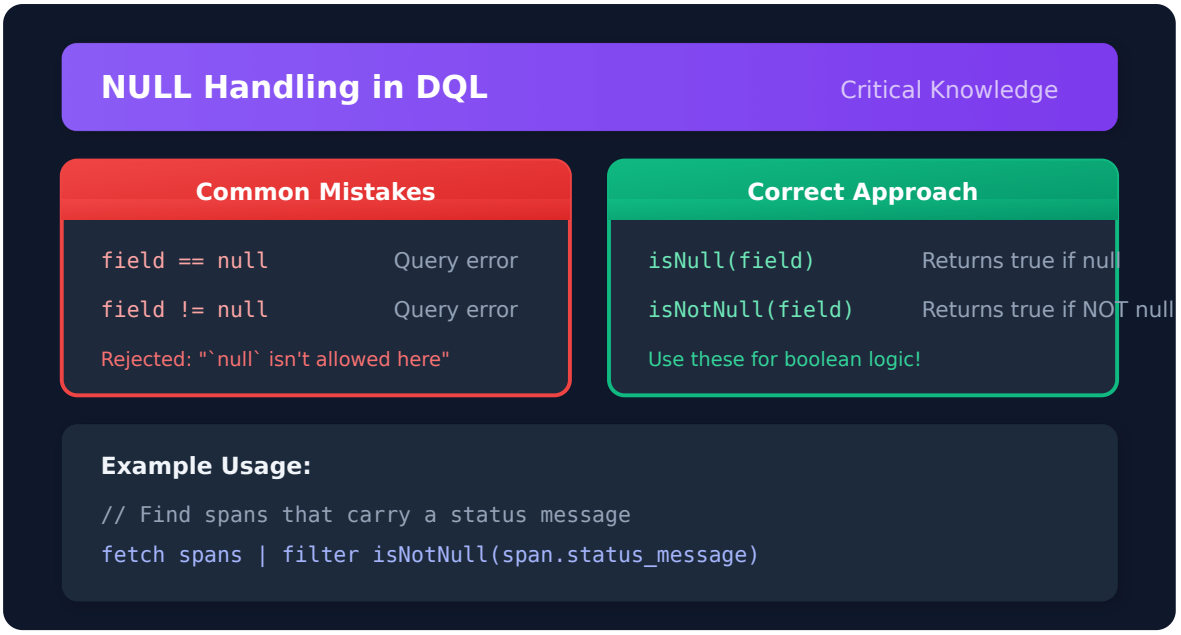
```
// Find slow database queries (over 100ms)
fetch spans, from:-1h
| filter isNotNull(db.system)
      and duration &gt; 100ms
| fieldsAdd duration_ms = duration / 1ms
| fields start_time,
         dt.service.name,
         db.system,
         db.operation.name,
         db.query.text,
         duration_ms
| sort duration_ms desc
| limit 50
```

```
// Summarize database usage
fetch spans, from:-1h
| filter isNotNull(db.system)
| summarize {
    db_span_count = count(),
    avg_duration_ms = avg(duration) / 1ms
  }, by: {db.system, db.namespace}
| sort db_span_count desc
```

---

9. Working with NULL Values

⚠️ **DQL has no `== null`.** Comparing a field with the `null` literal is rejected outright, so null checks always go through `isNull()` / `isNotNull()`. A comparison with a field that happens to be null (`span.status_code != "error"` on a span with no status) does not return true either — test for the null case explicitly.




```
// Find spans that have database information
fetch spans, from:-1h
| filter isNotNull(db.system)
| summarize {db_span_count = count()}, by: {db.system, db.namespace}
| sort db_span_count desc
```

```
// Find HTTP spans with missing route (potential instrumentation issue)
fetch spans, from:-1h
| filter isNotNull(http.request.method) and isNull(http.route)
| fields dt.service.name, span.name, http.request.method, url.path
| dedup dt.service.name, span.name
| limit 20
```

---

10. Combining Multiple Filters

Build complex queries by combining multiple filter conditions:

> 💡 **Performance Tip:** Apply more restrictive filters first for better query performance.

```
// Complex filter: Find slow SERVER spans in the checkout service
fetch spans, from:-1h
| filter dt.service.name == "checkout"
      and span.kind == "server"
      and duration &gt; 500ms
| fieldsAdd duration_ms = duration / 1ms
| fields start_time,
         span.name,
         duration_ms,
         span.status_code
| sort duration_ms desc
| limit 50
```

```
// Find error spans for specific services and operations
fetch spans, from:-1h
| filter span.status_code == "error"
      and in(dt.service.name, {"payment", "checkout"})
      and span.kind == "server"
| fieldsAdd duration_ms = duration / 1ms
| fields start_time,
         dt.service.name,
         span.name,
         span.status_message,
         trace.id,
         duration_ms
| sort start_time desc
| limit 50
```

```
// Find spans: either server errors OR slow successful requests
fetch spans, from:-1h
| filter http.response.status_code &gt;= 500
      or (http.response.status_code &gt;= 200
          and http.response.status_code &lt; 300
          and duration &gt; 1s)
| fieldsAdd duration_ms = duration / 1ms
| fields start_time,
         dt.service.name,
         http.request.method,
         http.route,
         http.response.status_code,
         duration_ms
| sort start_time desc
| limit 100
```

11. Request Attributes on Spans

Request attributes configured in your environment surface on spans under two templated field families. Both are `stable` in the semantic dictionary, and the difference between them is not visible in the name:

Field	Scope	Values	Type
`request_attribute.<name>`	Request	Reconciled for the request as a whole	**array**
`captured_attribute.<name>`	Span	Raw, as captured on that span	**array**

Use `request_attribute.<name>` when you want the attribute's settled value for a request; use `captured_attribute.<name>` when you need what an individual span actually captured before reconciliation.

> ⚠️ **Both are arrays.** `filter request_attribute.order_id == "A-1"` matches **nothing** — and fails silently, with no error. Use `matchesValue()` for membership, or an array function to reach an element.

Two further traps:

- **Mixed capture types collapse to string.** If the captured values have mixed types, all of them are converted to string and stored as a string array — so a numeric comparison that works in one environment can stop matching in another.
- **The field name is yours.** `<name>` is whatever you called the request attribute in its configuration, so these fields are not discoverable by browsing the dictionary. Read them from the request-attribute configuration, or from a span that carries them.

For the request-attribute question in its wider context — including what happened to the classic PurePath sub-timings and why external calls carry no endpoint — see **FAQ-22**.

> **Sources:** Request attributes (DT docs), Semantic Dictionary (DT docs). Field names, `stable` levels, array typing, and the mixed-type-to-string rule read from `dt.semantic_dictionary.fields` on a live SaaS tenant, 08/04/2026.

```
// Confirm the contract for both request-attribute field families
// before writing a filter. Scans zero billable bytes.
fetch dt.semantic_dictionary.fields
| filter contains(name, "request_attribute")
    or contains(name, "captured_attribute")
| fields name, stability, type, description
```

Summary

In this notebook, you learned:

✅ **DQL ≠ SQL** - Critical syntax differences (arrays use `{}`, use `==`, `isNull()`)
✅ **Filter by service** using exact match, `in()`, and pattern matching
✅ **Filter by span kind** - values are lowercase (`"server"`, `"client"`)
✅ **String matching** - `contains()`, `startsWith()`, `endsWith()`, `matchesPhrase()`
✅ **Find traces** by trace.id and identify root spans
✅ **Query HTTP spans** including status codes, methods, routes
✅ **Analyze database spans** to find slow queries
✅ **Handle NULL values** with `isNull()` / `isNotNull()`
✅ **Use** `dedup` to see unique values
✅ **Combine filters** for complex, precise queries
✅ **Query request attributes** — `request_attribute.<name>` and `captured_attribute.<name>` are arrays, not scalars

---

Next Steps

Continue to **SPANS-03: Trace Analysis & Troubleshooting** to learn:
- Identifying error patterns and failure points
- Latency analysis across services
- Root cause analysis techniques
- Tracing request flows through your system

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.